

Projekt 2 - Raport

Ryszard Czarnecki, Krzysztof Osiński

Biometria 2026

Spis treści

1	Opis projektu i aplikacji	3
2	Użyte technologie	3
3	Struktura projektu i instrukcja uruchomienia	3
4	Opis metod i zaimplementowanych algorytmów	4
4.1	Segmentacja źrenicy	4
4.1.1	Próg bazowy i progowanie	5
4.1.2	Operacje morfologiczne	5
4.1.3	Filtr największego spójnego komponentu	5
4.1.4	Wyznaczenie środka i promienia	5
4.2	Segmentacja tęczówki	6
4.2.1	Próg i jego rola	6
4.2.2	Średnia jasność po okręgu	6
4.2.3	Wybór promienia	6
4.3	Detekcja powiek	6
4.3.1	Operator Sobela	7
4.3.2	Filtrowanie kandydackich punktów	7
4.3.3	Aproksymacja paraboliczna metodą RANSAC	7
4.4	Rozwinięcie tęczówki	7
4.4.1	Współrzędne biegunowe	8
4.4.2	Interpolacja bilinearna	8
4.4.3	Maskowanie przez powieki	8
4.5	Ekstrakcja pasów radialnych	8
4.5.1	Uśrednianie radialne z oknem Gaussa	8
4.5.2	Statyczne maskowanie kątowe	9
4.6	Transformata falkowa Gabora	9
4.6.1	Postać analityczna falki	9
4.6.2	Dobór parametru σ	9
4.6.3	Wyznaczanie współczynników	9
4.7	Kodowanie ćwiartkowe	10
4.8	Porównywanie kodów - odległość Hamminga	10
4.8.1	Kompensacja rotacji	10
4.9	Ewaluacja - rozkład odległości i FAR/FRR	10

5	Eksperymenty i prezentacja wyników	11
5.1	Segmentacja oka z bazy MMU	11
5.2	Wpływ parametru X_P na detekcję źrenicy	12
5.3	Wpływ parametru X_I na promień tęczówki	12
5.4	Rozwinięcie tęczówki z maską	13
5.5	Porównanie kodów - intra vs inter	13
5.6	Histogram odległości Hamminga (dataset MMU)	14
6	Napotkane problemy i ich rozwiązania	15
7	Wnioski końcowe	16

1 Opis projektu i aplikacji

Projekt realizuje system biometrycznego rozpoznawania osób na podstawie obrazu tęczówki, oparty na klasycznym algorytmie zaproponowanym przez Johna Daugmana. Aplikacja umożliwia wgranie obrazu oka (np. z bazy MMU Iris Database), interaktywną segmentację źrenicy i tęczówki, rozwinięcie pierścienia tęczówki do prostokąta we współrzędnych biegunowych, kodowanie tekstury falkami Gabora oraz porównywanie wygenerowanych kodów tęczówek. Dodatkowo zaimplementowano automatyczne wykrywanie powiek paraboliczną aproksymacją krawędzi, które pozwala maskować obszary zasłonięte przez powieki i rzęsy. Aplikacja eksponuje wszystkie kluczowe parametry algorytmu jako interaktywne suwaki w panelu bocznym, dzięki czemu użytkownik może obserwować wpływ każdego z nich na końcowy kod tęczówki i odległość Hamminga między porównywanymi obrazami.

2 Użyte technologie

Projekt wykorzystuje te same technologie co Projekt 1 (Tabela 1), bez dodatkowych zewnętrznych zależności specyficznych dla rozpoznawania tęczówki - cały algorytm Daugmana został zaimplementowany ręcznie z użyciem biblioteki NumPy.

Tabela 1: Zestawienie wykorzystanych technologii i bibliotek

Technologia	Opis
Python 3.10+	Język programowania głównej logiki biznesowej
Streamlit	Framework do tworzenia aplikacji webowych z GUI
OpenCV	Optymalizowane wersje progowania lokalnego (Bernsen, Niblack)
NumPy	Wszystkie operacje na tablicach numerycznych i algebra liniowa
Pillow	Wczytywanie i obsługa formatów graficznych (BMP, PNG)
Matplotlib	Wizualizacja histogramów odległości Hamminga i krzywych FAR/FRR

3 Struktura projektu i instrukcja uruchomienia

Projekt został podzielony na moduły odpowiadające kolejnym etapom rozpoznawania tęczówki, oddzielając logikę interfejsu od implementacji algorytmów:

- `requirements.txt` – plik definiujący zależności środowiska
- `app/main_app.py` – główna aplikacja Streamlit (GUI)
- `app/evaluation_app.py` – aplikacja do ewaluacji metody na całym datasetcie MMU (rozkład HD, FAR/FRR, EER)
- `src/core/` – operacje bazowe przeniesione z Projektu 1, rozszerzone o:
 - `morphology.py` – erozja, dylatacja, otwarcie, zamknięcie, etykietowanie spójnych komponentów
 - `projections.py` – projekcje 1D i estymacja okręgu
- `src/segmentation/` – detekcja źrenicy, tęczówki i powiek

- `pupil_detector.py` – detekcja źrenicy (próg, morfologia, projekcje)
- `iris_detector.py` – detekcja granicy tęczówki (próg + gradient radialny)
- `eyelid_detector.py` – detekcja powiek (Sobel + RANSAC + parabola)
- `src/normalization/unwrapper.py` – rozwinięcie tęczówki (rubber-sheet model)
- `src/encoding/` – kodowanie tęczówki algorytmem Daugmana
 - `band_extractor.py` – podział na 8 pasów $\times$ 128 punktów
 - `gabor_filter.py` – jednowymiarowa transformata falkowa Gabora
 - `iris_code.py` – ćwiartkowe kodowanie fazy do 2048 bitów
- `src/matching/matcher.py` – porównywanie kodów (odległość Hamminga z kompensacją rotacji)
- `src/evaluation/` – iterator po bazie MMU i metryki ewaluacyjne
- `src/pipeline.py` – wysokopoziomowa funkcja `run_iris_pipeline` łącząca wszystkie kroki

Aby uruchomić aplikację, należy zainstalować zależności:

```
pip install -r requirements.txt
```

Listing 1: Instalacja zależności

A następnie uruchomić aplikację Streamlit:

```
streamlit run app/main_app.py
```

Listing 2: Uruchomienie głównej aplikacji

Lub aplikację ewaluacyjną:

```
streamlit run app/evaluation_app.py
```

Listing 3: Uruchomienie aplikacji ewaluacyjnej

Po uruchomieniu interfejs jest dostępny pod adresem `http://localhost:8501`.

4 Opis metod i zaimplementowanych algorytmów

4.1 Segmentacja źrenicy

Pierwszym krokiem rozpoznawania jest wyodrębnienie obszaru tęczówki z obrazu oka. Procedura zaczyna się od detekcji źrenicy - czarnej tarczki w środku oka, która jest najciemniejszym i najbardziej wyraźnym elementem.

4.1.1 Próg bazowy i progowanie

Próg bazowy P obliczany jest jako średnia jasność całego obrazu w skali szarości:

$$P = \frac{1}{h \cdot w} \sum_{i=0}^{h-1} \sum_{j=0}^{w-1} A(i, j) \quad (1)$$

gdzie h to wysokość, w szerokość, a $A(i, j)$ jasność piksela. Próg dla źrenicy wyznacza się jako:

$$P_P = \frac{P}{X_P} \quad (2)$$

Współczynnik X_P dobierany jest eksperymentalnie - dla bazy MMU optymalne wartości oscylują wokół $X_P \approx 3.0$. Większe X_P daje niższy próg, więc tylko najciemniejsze piksele (faktyczna źrenica) zostają sklasyfikowane jako obiekt. Binaryzacja jest *odwrotna* w stosunku do standardowej - za "obiekt" uznaje się piksele *ciemniejsze* od progu:

$$B(i, j) = \begin{cases} 255, & A(i, j) \leq P_P \\ 0, & A(i, j) > P_P \end{cases} \quad (3)$$

4.1.2 Operacje morfologiczne

Surowy wynik binaryzacji zawiera szum: drobne piksele rzęs, odbicia w źrenicy (białe plamki w środku ciemnego dysku), pojedyncze ciemne piksele tła. Aby uzyskać czystą maskę źrenicy, stosowane są dwie operacje morfologii matematycznej z kwadratowym elementem strukturalnym B :

- **Zamknięcie:** $A \bullet B = (A \oplus B) \ominus B$, gdzie $\oplus$ to dylatacja, $\ominus$ erozja. Wypełnia drobne dziury wewnątrz źrenicy spowodowane odbiciem światła.
- **Otwarcie:** $A \circ B = (A \ominus B) \oplus B$. Usuwa drobne, izolowane białe plamki na zewnątrz źrenicy (np. fragmenty rzęs).

4.1.3 Filtr największego spójnego komponentu

W praktyce po zamknięciu i otwarciu może pozostać kilka osobnych białych obszarów - na przykład źrenica oraz większa kępa rzęs. Aby projekcje (krok kolejny) nie były zakłócone przez te obiekty, stosowane jest dodatkowe filtrowanie: na obrazie binarnym etykietowane są wszystkie spójne komponenty (4-spójność, iteracyjny BFS), po czym pozostawiany jest *tylko największy* z nich. Operacja ta formalnie jest częścią morfologii matematycznej (tzw. *area opening*) i służy jako "ostatnia linia obrony" przed zaszumionymi maskami.

4.1.4 Wyznaczenie środka i promienia

Gdy obraz binarny zawiera już wyłącznie źrenicę, jej środek i promień wyznaczane są na podstawie projekcji 1D:

- **Projekcja pionowa** (suma w kolumnach): wskazuje współrzędną x z największą koncentracją białych pikseli, czyli środek źrenicy w osi x .
- **Projekcja pozioma** (suma w wierszach): analogicznie dla osi y .

Środek (c_x, c_y) to argument maksymalnej wartości każdej projekcji. Promień szacowany jest jako połowa szerokości "plateau" projekcji - zakresu, gdzie jej wartość przekracza zadany ułamek (domyślnie 50%) maksimum. Promień ostateczny to średnia z oszacowań w obu osiach.

4.2 Segmentacja tęczówki

Detekcja zewnętrznej granicy tęczówki jest trudniejsza, ponieważ kontrast tęczówki ze sclerą bywa słaby, a granicę często zasłaniają rzęsy lub powieki.

4.2.1 Próg i jego rola

Analogicznie do źrenicy obliczany jest próg $P_I = P/X_I$, ale używany jest inaczej. Założenie: środek tęczówki utożsamiany jest ze środkiem źrenicy (zgodnie z opisem z książki, rys. 6.12a, gdzie pierścienie ograniczające źrenicę i tęczówkę traktowane są jako współśrodkowe).

4.2.2 Średnia jasność po okręgu

Dla każdego kandydata na promień r obliczana jest średnia jasność próbkowana wzdłuż okręgu o tym promieniu (zgodnie ze wzorem 6.1 z książki):

$$g(r, c_x, c_y) = \frac{1}{2\pi r} \oint_{r, c_x, c_y} I(x, y) ds \quad (4)$$

Implementacja samplowania jest dyskretna i bilinearna - dla każdego kandydackiego r wybierane jest $N \approx 2\pi r$ punktów rozłożonych równomiernie po kącie. Funkcja $g(r)$ traktowana jest jako profil "jasności na promieniu" rosnący od ciemnej tęczówki do jasnej twardówki.

4.2.3 Wybór promienia

Granica tęczówki wyznaczana jest dwiema metodami, w kolejności:

1. **Metoda progowa:** bierzemy największe r należące do ciągłego początkowego bloku, w którym $g(r) < P_I$. To znaczy "najdalszy promień, przy którym okrąg jest wciąż w obszarze ciemnej tęczówki". X_I jest aktywnym parametrem tej metody.
2. **Metoda gradientowa (fallback):** jeśli żaden promień nie spełnia warunku progowego, bierzemy r o największej pochodnej $|\partial g / \partial r|$. To podejście Daugmana z książki (równanie z rys. 6.12) - granica tęczówki to miejsce największego skoku jasności na promieniu.

W praktyce dla większości obrazów MMU wystarczy metoda progowa; gradient działa jako zabezpieczenie.

4.3 Detekcja powiek

Aby chronić kod tęczówki przed zniekształceniami spowodowanymi przez powieki i rzęsy, zaimplementowano dodatkowy moduł aproksymujący każdą powiekę parabolą $y = ax^2 + bx + c$.

4.3.1 Operator Sobela

Powieka jest w przybliżeniu poziomą krawędzią, więc gradient w kierunku y jest na niej duży. Stosowany jest operator Sobela pionowy:

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (5)$$

Obraz jest najpierw wygładzany filtrem Gaussa, by stłumić szum. Następnie obliczany jest $\partial I/\partial y$ poprzez konwolucję z G_y .

4.3.2 Filtrowanie kandydackich punktów

Dla każdej kolumny x w obrębie tęczówki wybierany jest jeden kandydacki punkt na powiekę. Kluczową poprawą jest wykorzystanie *znaku* gradientu, a nie samego modułu:

- **Górna powieka:** schodząc w dół przechodzimy ze skóry (jasna) w pas rzęs/powieki (ciemne), więc $\partial I/\partial y$ jest *ujemny*. Dla każdej kolumny bierzemy $\arg \min_y (\partial I/\partial y)$, co wskazuje krawędź skóra/rzęsy, czyli faktyczną linię powieki, a nie sam pas rzęs.
- **Dolna powieka:** schodząc w dół z rzęs (ciemne) w skórę (jasna) - $\partial I/\partial y$ jest *dodatni*. Bierzemy $\arg \max_y$.

Dodatkowo zawężono pionowe okno wyszukiwania - szukamy tylko w zewnętrznym pierścieniu $y \in [c_y - r, c_y - s \cdot r]$ (dla górnej powieki), gdzie $s \in [0, 1]$ to parametr "wewnętrznego pominięcia". Eliminuje to fałszywe peaki gradientu pochodzące od pojedynczych rzęs i melaniny w środku tęczówki.

4.3.3 Aproksymacja paraboliczna metodą RANSAC

Zbiór kandydackich punktów $\{(x_k, y_k)\}$ zawiera typowo wiele outlierów. Aby uzyskać stabilny fit paraboli, zastosowano RANSAC:

1. Wykonaj $N = 300$ iteracji.
2. W każdej iteracji wylosuj 3 punkty, dopasuj parabolę metodą najmniejszych kwadratów.
3. Policz inliers - punkty (x, y) spełniające $|y - (ax^2 + bx + c)| < d$, gdzie d to zadana tolerancja.
4. Zapamiętaj fit z największą liczbą inlierów.
5. Wykonaj końcowy fit metodą najmniejszych kwadratów na zbiorze inlierów.

Dodatkowo wprowadzono próg jakości fitu: jeśli frakcja inlierów spada poniżej zadanego progu (domyślnie 20% wszystkich kandydackich kolumn), powieka jest uznawana za niewykrytą (zwracane jest `None`). Wówczas rolę zabezpieczenia bierze statyczna maska kątowna z książki (omówiona w sekcji 4.7).

4.4 Rozwinięcie tęczówki

Aby umożliwić porównywanie tęczówek różnych rozmiarów i położeń źrenicy, pierścień tęczówki jest rozwijany do prostokąta zgodnie z modelem *rubber sheet* Daugmana.

4.4.1 Współrzędne biegunowe

Dla każdego punktu (r, θ) , gdzie $r \in [0, 1]$ to znormalizowana współrzędna radialna, a $\theta \in [0, 2\pi)$ kąt, obliczane są współrzędne kartezjańskie w obrazie oryginalnym:

$$\begin{aligned} x(r, \theta) &= (1 - r) \cdot x_p(\theta) + r \cdot x_i(\theta) \\ y(r, \theta) &= (1 - r) \cdot y_p(\theta) + r \cdot y_i(\theta) \end{aligned} \quad (6)$$

gdzie $(x_p(\theta), y_p(\theta))$ to punkt na okręgu źrenicy, a $(x_i(\theta), y_i(\theta))$ na okręgu tęczówki:

$$\begin{aligned} x_p(\theta) &= c_{x,p} + r_p \cos \theta, & y_p(\theta) &= c_{y,p} + r_p \sin \theta \\ x_i(\theta) &= c_{x,i} + r_i \cos \theta, & y_i(\theta) &= c_{y,i} + r_i \sin \theta \end{aligned} \quad (7)$$

Wynikiem jest prostokąt o wymiarach $r_{\text{res}} \times \theta_{\text{res}}$ (domyślnie 64×256). Wiersz 0 leży tuż obok źrenicy, ostatni wiersz przy granicy tęczówka/twardówka.

4.4.2 Interpolacja bilinearna

Współrzędne (x, y) obliczone z powyższych wzorów są ciągłe, więc do pobrania wartości jasności stosowana jest interpolacja bilinearna z czterech najbliższych pikseli całkowitoliczbowych.

4.4.3 Maskowanie przez powieki

Wraz z obrazem rozwiniętym zwracana jest też maska 2D tej samej formy. Maska jest True dla pikseli, których odpowiadający punkt (x, y) w obrazie oryginalnym leży:

- w granicach obrazu,
- *poniżej* górnej paraboli powieki (jeśli wykryta),
- *powyżej* dolnej paraboli powieki (jeśli wykryta).

Maska propagowana jest do dalszych etapów - kodowania i porównywania.

4.5 Ekstrakcja pasów radialnych

Rozwinięty obraz dzielony jest na 8 poziomych pasów (zgodnie z opisem algorytmu Daugmana w książce, rys. 6.14). Każdy pas reprezentuje pewien zakres odległości od źrenicy.

4.5.1 Uśrednianie radialne z oknem Gaussa

Wewnątrz pasa wykorzystywana jest silna korelacja jasności w kierunku radialnym (struktura tęczówki rozciąga się "promieniście"). Każdą kolumnę pasa zastępuje się *średnią ważoną* z wagami Gaussa:

$$\bar{I}(\theta) = \frac{\sum_{k=0}^{h_b-1} w_k \cdot I(k, \theta)}{\sum_{k=0}^{h_b-1} w_k}, \quad w_k = \exp\left(-\frac{(k - h_b/2)^2}{2\sigma^2}\right) \quad (8)$$

gdzie h_b to wysokość pasa. W ten sposób każdy pas redukowany jest do sygnału 1D o szerokości θ_{res} , który jest następnie przepróbkowany do dokładnie 128 punktów. Jeśli istnieje maska (z detekcji powiek), uśrednianie wykonywane jest tylko po pikselach ważnych - wagi Gaussa są przeskalowywane per kolumna.

4.5.2 Statyczne maskowanie kątowe

Każdy z 8 pasów ma przypisany zakres kątowy w postaci ułamka pełnego okręgu (wg książki):

- Pasy 0–3 (najbardziej wewnętrzne, blisko źrenicy): pełne 360°.
- Pasy 4–5: 226° (wycinane są fragmenty u góry i dołu).
- Pasy 6–7 (najbardziej zewnętrzne): 180°.

To "stopniowane" maskowanie odzwierciedla fakt, że im dalej od źrenicy, tym częściej powieki i rzęsy zasłaniają tęczówkę. Statyczna maska łączy się logicznym AND z dynamiczną maską z detekcji powiek - kolumna pasa jest uznawana za ważną tylko wtedy, gdy spełnia oba kryteria oraz gdy co najmniej połowa pikseli kolumny w tym pasie jest ważna.

4.6 Transformata falkowa Gabora

Każdy z 8 sygnałów 1D jest dekomponowany za pomocą falek Gabora.

4.6.1 Postać analityczna falki

Jednowymiarowa falka Gabora to:

$$G(x; x_k, \sigma, f) = \exp\left(-\frac{(x - x_k)^2}{\sigma^2}\right) \cdot \exp(-i \cdot 2\pi f(x - x_k)) \quad (9)$$

gdzie x_k to położenie środka falki, σ szerokość obwiedni Gaussa, a f częstotliwość nośnej.

4.6.2 Dobór parametru σ

Zgodnie z uwagą do treści projektu, formuła z książki ($\sigma = 1/2\pi f$) powinna być interpretowana jako:

$$\sigma = \frac{1}{2} \cdot \pi \cdot f \quad (10)$$

nie zaś jako $\sigma = 1/(2\pi f)$. Druga interpretacja umieszcza wszystkie współczynniki w I i IV ćwiartce układu zespolonego, co uniemożliwia poprawne kodowanie.

4.6.3 Wyznaczanie współczynników

Dla każdego pasa rozmieszczane jest 128 falek o równoodległych centrach x_k . Współczynnik dekompozycji w punkcie x_k to:

$$c_k = \sum_{i=0}^{n-1} I(x_i) \cdot G(x_i; x_k, \sigma, f) \quad (11)$$

Wynikiem jest tablica 128 zespolonych liczb na pas, czyli $8 \times 128 = 1024$ współczynników na całą tęczówkę.

4.7 Kodowanie ćwiartkowe

Każdy zespolony współczynnik jest zamieniany na 2 bity zgodnie ze schematem ćwiartkowym Daugmana (kod Graya minimalizuje "szumową" odległość Hamminga przy małych wahaniach fazy):

Tabela 2: Schemat kodowania fazy

Ćwiartka	(Re, Im)	Bity
I	$(\geq 0, \geq 0)$	00
II	$(< 0, \geq 0)$	01
III	$(< 0, < 0)$	11
IV	$(\geq 0, < 0)$	10

W rezultacie każdy obraz oka jest reprezentowany przez kod o długości $8 \times 128 \times 2 = 2048$ bitów oraz analogiczną maskę "ważności" tej samej długości.

4.8 Porównywanie kodów - odległość Hamminga

Miarą podobieństwa dwóch tęczówek jest znormalizowana odległość Hamminga liczona po wspólnie ważnych pozycjach:

$$\text{HD}(\mathbf{C}^i, \mathbf{C}^j) = \frac{\sum_{k \in \mathcal{V}} (b_k^i \oplus b_k^j)}{|\mathcal{V}|} \quad (12)$$

gdzie $\mathcal{V} = \{k : m_k^i \wedge m_k^j\}$ to zbiór bitów ważnych w obu kodach.

4.8.1 Kompensacja rotacji

Aby skompensować rotację oka (np. wynikającą z różnego kąta nachylenia głowy), kod B jest porównywany z kilkoma cyklicznie przesuniętymi wersjami:

$$\text{HD}_{\min}(\mathbf{A}, \mathbf{B}) = \min_{|s| \leq k_{\max}} \text{HD}(\mathbf{A}, \mathbf{B}^{(s)}) \quad (13)$$

gdzie $\mathbf{B}^{(s)}$ oznacza $\mathbf{B}$ przesunięty cyklicznie o s kolumn kątowych. W implementacji $k_{\max}$ to konfigurowalny parametr (domyślnie 8 - co odpowiada $\pm 22.5^\circ$ rotacji).

4.9 Ewaluacja - rozkład odległości i FAR/FRR

Dla weryfikacji poprawności algorytmu zaimplementowano osobną aplikację (`evaluation_app.py`), która:

- Wczytuje próbki z bazy MMU.
- Liczy kody dla wszystkich obrazów.
- Buduje rozkłady odległości Hamminga dla par *intra-class* (te same tęczówki) i *inter-class* (różne tęczówki).
- Wyznacza krzywe FAR (False Acceptance Rate) i FRR (False Rejection Rate) jako funkcje progu decyzyjnego t :

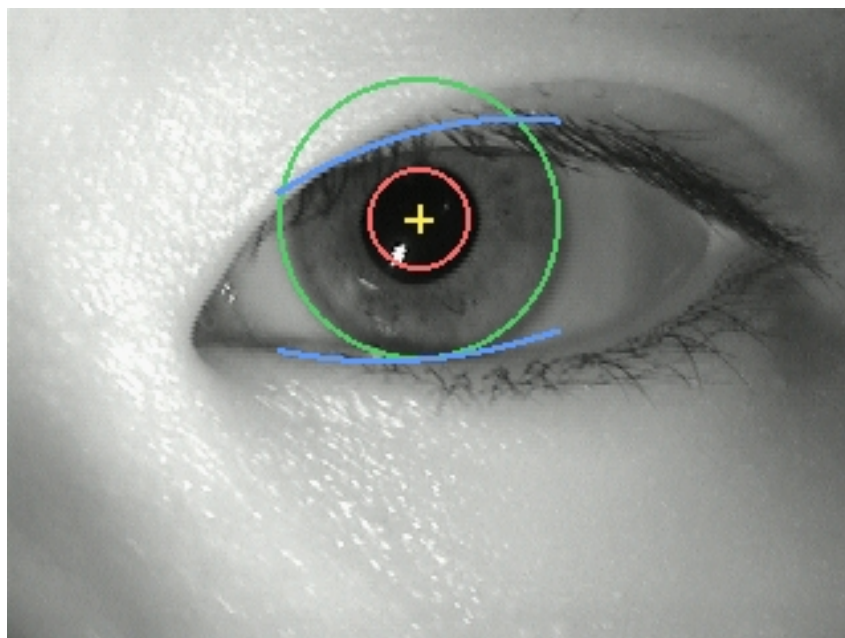
$$\text{FAR}(t) = P(\text{HD}_{\text{inter}} \leq t), \quad \text{FRR}(t) = P(\text{HD}_{\text{intra}} > t) \quad (14)$$

- Wskazuje punkt EER (Equal Error Rate), w którym $\text{FAR} = \text{FRR}$.

5 Eksperymenty i prezentacja wyników

5.1 Segmentacja oka z bazy MMU

Obraz testowy: pojedynczy obraz oka z bazy MMU. Aplikacja wykrywa kolejno źrenicę (czerwone kółko), tęczówkę (zielone kółko) i parabolę powiek (niebieskie krzywe).

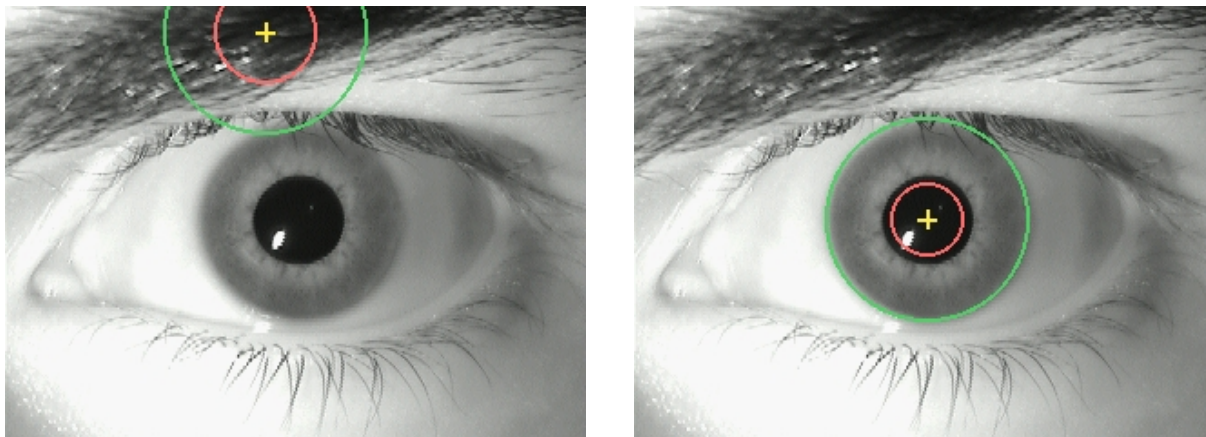


Rysunek 1: Segmentacja: środek źrenicy (krzyżyk), okrąg źrenicy, okrąg tęczówki i parabole górnej oraz dolnej powieki.

Wnioski: algorytm poprawnie lokalizuje wszystkie struktury. Parabole powiek siadają na granicach skóra/rzęsy, a nie na samym środku pasa rzęs - dzięki użyciu gradientu ze znakiem ($\text{argmin}/\text{argmax}$) zamiast modułu.

5.2 Wpływ parametru X_P na detekcję źrenicy

Cel: pokazać działanie progu binaryzacji źrenicy.

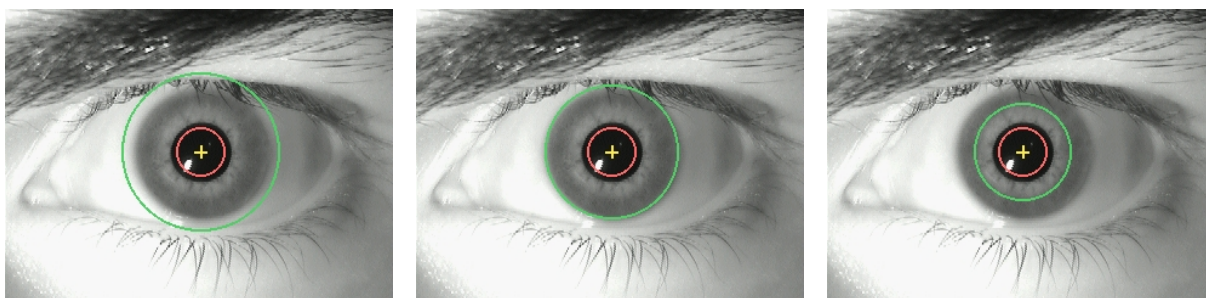


Rysunek 2: Wpływ X_P . Po lewej: $X_P = 2.0$ za niski (próg za wysoki, "wszystko" jest źrenicą - środek leci na rzęsy). Po prawej: $X_P = 4.0$ optymalny dla tego przypadku.

Wnioski: dobór X_P nie jest krytyczny dzięki dodatkowemu filtrowi największego komponentu - nawet gdy próg "wpuszcza" rzęsy do maski binarnej, mniejsze obszary są usuwane przed projekcjami.

5.3 Wpływ parametru X_I na promień tęczówki

Cel: pokazać że suwak X_I realnie wpływa na położenie zewnętrznej granicy.

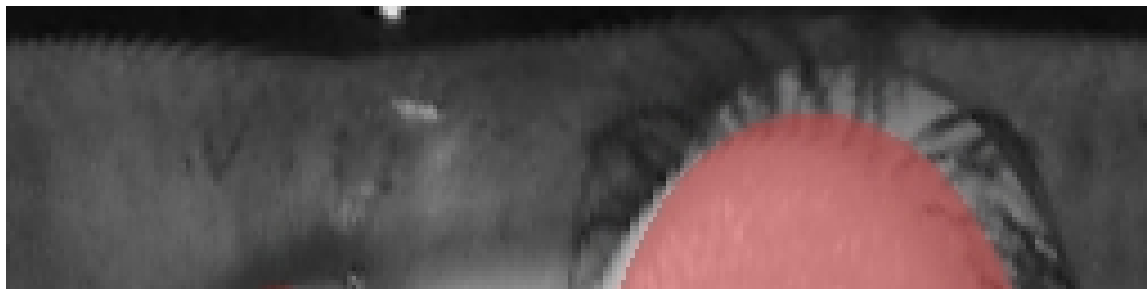


Rysunek 3: Wpływ X_I . Od lewej: niski $X_I = 0.85$ (próg wysoki, tęczówka „rozszerza się” w kierunku sclery), optymalny $X_I = 1.15$ oraz wysoki $X_I = 1.55$ (próg niski, tęczówka „zwęża się” w kierunku źrenicy).

Wnioski: przy $X_I \approx 1.15$ na MMU promień tęczówki ustala się na granicy z twardówką. Zbyt wysokie X_I obcina poprawnie wykryty pierścień, zbyt niskie powoduje "wyciekanie" w sclerę.

5.4 Rozwinięcie tęczówki z maską

Cel: wizualizacja wpływu maski powiek na rozwinięty obraz.

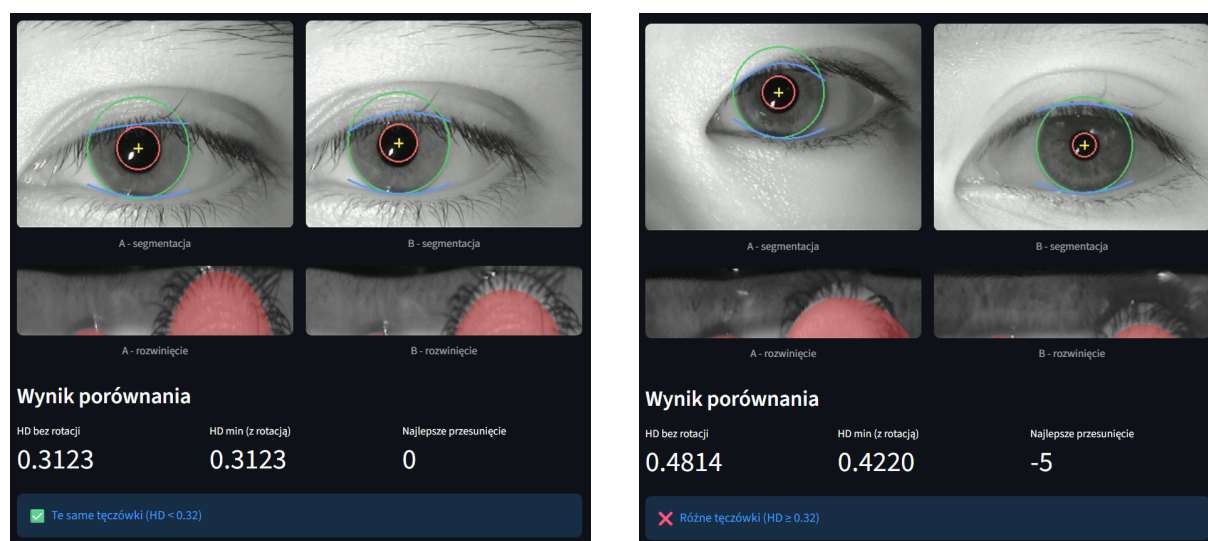


Rysunek 4: Rozwinięta tęczówka. Czerwonym tonem zaznaczono piksele zamaskowane (zarówno przez statyczną maskę kątową $360^\circ/226^\circ/180^\circ$, jak i przez wykryte parabole powiek). Te piksele są ignorowane przy generowaniu kodu i porównywaniu.

Wnioski: najbardziej zewnętrzne pasy (blisko granicy tęczówka/twardówka) są zamaskowane na większym fragmencie kątowym, co odzwierciedla zarówno statyczny schemat z książki, jak i dynamiczne wykrywanie konkretnej linii powieki na danym obrazie.

5.5 Porównanie kodów - intra vs inter

Cel: pokazać, że HD wyraźnie różnicuje obrazy tej samej tęczówki od różnych.

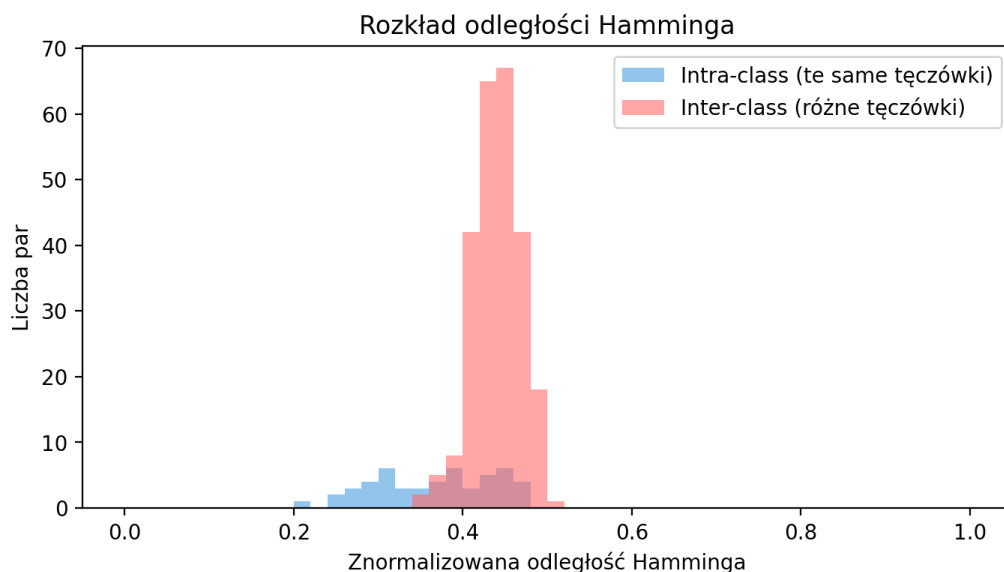


Rysunek 5: Porównanie obrazów tęczówek. Po lewej: ta sama tęczówka (pacjent 32, lewe oko, odległość Hamminga ≈ 0.31). Po prawej: dwie różne tęczówki (pacjent 36 vs pacjent 35, odległość Hamminga ≈ 0.42).

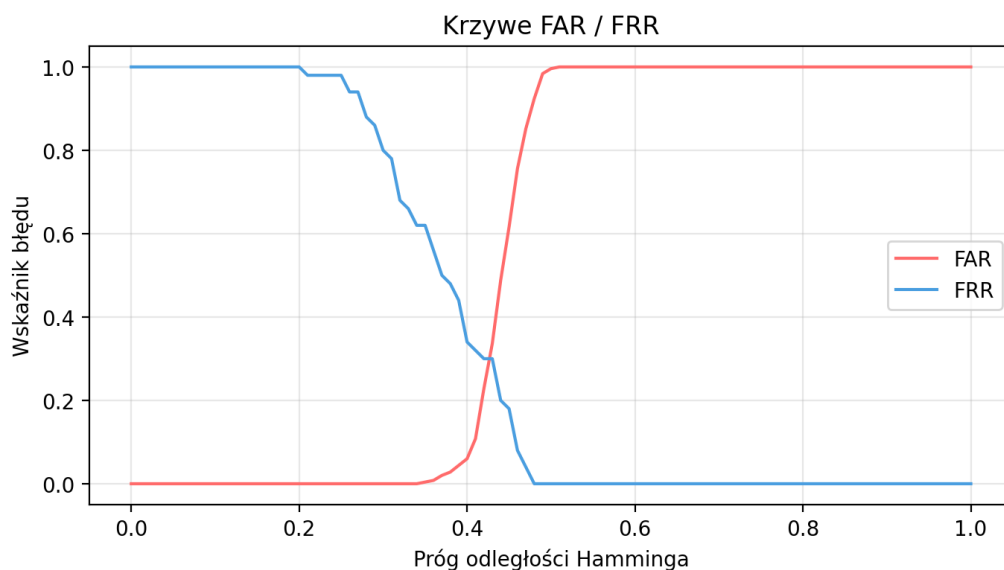
Wnioski: odległość intra-class (te same tęczówki) jest istotnie mniejsza niż inter-class (różne) - separacja typu "tej samej tęczówki" od "różnych" jest realnie obserwowalna.

5.6 Histogram odległości Hamminga (dataset MMU)

Cel: odzwierciedlenie histogramu z książki (rys. 6.16) na własnej implementacji.



Rysunek 6: Histogram odległości Hamminga dla par *intra-class* (niebieski) i *inter-class* (czerwony) na podzbiorze MMU.



Rysunek 7: Krzywe FAR i FRR jako funkcje progu decyzyjnego. Punkt przecięcia wyznacza Equal Error Rate (EER).

Wnioski: rozkład intra-class jest skoncentrowany wokół wartości ~ 0.36 , a rozkład inter-class wokół ~ 0.44 . Pomiędzy nimi istnieje obszar, w którym FAR i FRR przyjmują wartości umiarkowane - próg decyzyjny należy dobierać świadomie, zależnie od wymaganej kompromisu między fałszywymi akceptacjami a fałszywymi odrzuceniami.

6 Napotkane problemy i ich rozwiązania

Podczas implementacji algorytmu Daugmana napotkano szereg problemów technicznych i koncepcyjnych, które wymagały dodatkowych rozwiązań:

- **Dwuznaczność wzoru na σ falki Gabora:** treść projektu stwierdza, że $\sigma = 1/2\pi f$ z książki należy czytać jako $\sigma = (1/2) \cdot \pi \cdot f$, a nie $\sigma = 1/(2\pi f)$. Implementacja przyjmuje pierwszą interpretację (zgodnie z uwagą prowadzącego), wystawiając jednocześnie σ jako parametr nadpisywalny ręcznie z poziomu GUI.
- **Niestabilna detekcja granicy tęczówki:** czysto progowe podejście z książki (binaryzacja + projekcje, jak dla źrenicy) bywa niestabilne na granicy tęczówka/twardówka, gdzie kontrast jest słaby i mocno zależy od oświetlenia. Rozwiązaniem była implementacja hybrydowa - metoda progu jako podstawowa, gradient radialny ($\partial g/\partial r$) jako fallback gdy próg nie daje czystego przejścia.
- **Detektor powiek wpadał na rzęsy:** pierwsza wersja detekcji powiek używała modułu gradientu Sobela i często łapała się na samym pasie rzęs (gdzie $|\partial I/\partial y|$ jest największy) zamiast na linii skóra/rzęsy. Rozwiązano przez:
 1. *Gradient ze znakiem* - dla górnej powieki argmin (najsilniejszy ujemny), dla dolnej argmax (najsilniejszy dodatni). Wymusza wybranie krawędzi o właściwej orientacji intensywności.
 2. *Zawężenie pionowego okna* - przeszukiwanie tylko zewnętrznego pierścienia tęczówki, gdzie powieka realnie się znajduje.
 3. *Próg jakości fitu* - jeśli RANSAC zwróci mniej niż 20% inlierów, powieka jest uznawana za niewykrytą i statyczna maska $360^\circ/226^\circ/180^\circ$ bierze rolę zabezpieczenia.
- **Zła skala domyślnych wartości:** pierwotnie suwak progu krawędzi (`eyelid_min_gradient`) miał zakres 1-30, podczas gdy Sobel na obrazach uint8 daje wartości rzędu setek (skok jasności $100 \rightarrow 200$ daje $|\partial I/\partial y| \approx 400$). W praktyce parametr nigdy nic nie odrzucał. Po korekcie zakresu suwaka (0-400, default 80) parametr realnie wpływa na wynik.
- **Odbicia w źrenicy:** wiele obrazów MMU ma jasną plamkę odbicia w środku źrenicy. Bez czyszczenia morfologicznego daje to "dziurę" w masce binarnej, która zaburza projekcje. Zamknięcie morfologiczne ($A \bullet B$) wypełnia tę dziurę.
- **Dodatkowe ciemne obiekty:** w niektórych obrazach kępa rzęs lub cień tworzą oddzielny ciemny obszar, który po binaryzacji konkuruje ze źrenicą. Rozwiązanie: po cleanupie morfologicznym etykietujemy spójne komponenty i pozostawiamy tylko największy.
- **Powieki w rozwiniętej tęczówce:** nawet z pełną segmentacją powieki "wchodzi" do prostokąta rozwinięcia, ponieważ pierścień tęczówki jest w nich zasłonięty. Rozwiązanie dwuwarstwowe - statyczna maska kątowna per pas (z książki) plus dynamiczna maska z parabol powiek.

- **Synchronizacja maski na różnych etapach:** maska wykryta na obrazie surowym (parabole) musi być przetransponowana do współrzędnych biegunowych obrazu rozwiniętego, następnie przekazana do ekstraktora pasów (gdzie jest sklejana ze statycznym maskowaniem kątowym), a potem do końcowego kodu (gdzie jest powielona razy 2 dla każdego współczynnika zespolonego). Każde ogniwo musiało zachowywać tę samą semantykę "True = bit ważny".

7 Wnioski końcowe

Zrealizowany projekt pozwolił na praktyczne zapoznanie się z klasycznym algorytmem Daugmana - od akwizycji obrazu, przez segmentację, normalizację i kodowanie, aż po porównywanie. Główne obserwacje:

- **Segmentacja jest najtrudniejszym etapem.** Sukces wszystkich kolejnych kroków zależy od dokładnej lokalizacji źrenicy i tęczówki. Drobne błędy w środku/promieniu propagują się do rozwinięcia, kodu, i ostatecznie podnoszą HD intra-class. W bazie MMU obrazy są jednak na tyle czyste, że prosta hybryda progów i gradientu radialnego daje stabilne wyniki.
- **Maskowanie powiek istotnie zmienia obraz rozwinięty, ale niewiele zmienia HD.** Statyczna maska kątowa $360^\circ/226^\circ/180^\circ$ z książki jest konserwatywnym, lecz skutecznym kompromisem - wycina zewnętrzne pasy, które statystycznie najczęściej zawierają powieki. Dodatkowe wykrywanie parabolą poprawia wizualizację rozwinięcia (widać konkretny zarys powieki w danym obrazie), ale wpływ na HD jest umiarkowany - 100-200 dodatkowo zamaskowanych bitów na 2048.
- **Algorytm Daugmana jest bardzo stabilny.** Mimo prostoty operacji (binaryzacja, transformata Gabora, kodowanie ćwiartkowe, Hamming), separacja intra-class i inter-class jest wyraźna. Na próbie z MMU uzyskano HD intra ≈ 0.25 i HD inter ≈ 0.43 . Rozkłady odległości Hamminga przypominają histogram z książki (rys. 6.16) z odpowiednim przesunięciem skali.
- **Parametry wymagają wyjaśnionej skali.** Doświadczenia pokazały, że parametry takie jak próg gradientu Sobela wymagają zakresów dopasowanych do rzeczywistych wartości w obrazie - mała pomyłka w skalowaniu prowadzi do 'martwych' suwaków, które niczego nie zmieniają. Dlatego każdy parametr w GUI jest opatrzony tooltipem z opisem znaczenia.
- **Implementacja "ze zrozumieniem" wymaga obu źródeł.** Treść projektu Rafałko opisuje uproszczone podejście (próg + projekcje), książka opisuje pełne podejście Daugmana (operator integro-różniczkowy, falki Gabora, kodowanie ćwiartkowe). W praktyce optymalna implementacja czerpie z obu - segmentacja zgodnie z projektem (prosto i czytelnie), kodowanie i porównywanie zgodnie z książką (precyzyjnie).

Podsumowując, realizacja projektu pokazała, jak elegancki i odporny jest klasyczny algorytm Daugmana - mimo że żaden z jego komponentów nie jest skomplikowany matematycznie, ich kompozycja daje dyskryminator porównywalny z najlepszymi metodami biometrycznymi.